Informe Técnico de Corrección de Bugs y Avance Módulo 1

PubTube • Módulo 1 (Ingesta, Metadatos y Catálogo) **Fecha:** Octubre 2026 • Sprint 3 VALIDADO 100%

1. Resumen Ejecutivo

En atención al documento de auditoría externa (*Informe de Revisión de Código - Módulo 1 PubTube*), se ejecutó una intervención integral de corrección y aseguramiento de calidad sobre la base de código. Se priorizó resolver las vulnerabilidades críticas de seguridad, fallas de validación e inconsistencias de estado en la base de datos relacional y el almacenamiento en Garage (S3-compatible).

218 / 218	60 / 60	0	0
TESTS UNITARIOS (100% OK)	TESTS E2E / INTEGRACIÓN	VULNERABILIDADES DE SEGURIDAD	ERRORES / WARNINGS LINTER

2. Desglose de Bugs Corregidos

BUG-02 (CORR-06): Vulnerabilidad de Path Traversal en `filename` — CRÍTICA

Problema: El endpoint `POST /api/content/init` no validaba la presencia de caracteres de escape de directorio (como `../../etc/passwd`). Esto permitía alterar la clave de objeto en el almacenamiento S3/Garage, habilitando posibles escrituras fuera del namespace designado.

Solución: Se incorporó un validador que rechaza inmediatamente peticiones con secuencias `..`, `/` o `\` con código `400 Bad Request`, complementado con saneamiento forzoso mediante `path.basename`.

BUG-01 (CORR-01): Pérdida Silenciosa de Datos en DTOs de Subida — ALTA

Problema: El `ValidationPipe` global opera con `whitelist: true`. Al carecer `InitUploadDto` y `CompleteUploadDto` de decoradores de `class-validator`, NestJS descartaba silenciosamente todos los atributos del body (`filename`, `sizeBytes`, `mimeType`, `parts`), provocando fallos en cascada.

Solución: Se aplicaron decoradores de validación y transformación tipada (`@IsString`, `@IsNotEmpty`, `@IsIn`, `@IsInt`, `@Min`, `@Type`, `@ValidateNested` y expresiones regulares de hash SHA-256).

BUG-03 (CORR-03): Desconexión de Máquina de Estados y Metadatos Huérfanos — ALTA

Problema: El método `marcarComoListo` emitía el evento `metadata.updated` pero **no persistía** los metadatos en la tabla `metadata_versions` ni actualizaba `videos.status = 'listo'` en PostgreSQL, dejando la base de datos desincronizada del bus de eventos.

Solución: Se unificó la lógica bajo una transacción ACID bloqueante (`FOR UPDATE`) que inserta la versión 1 en `metadata_versions` y actualiza el estado a `listo`. Además, se sincronizó `actualizarMetadatos` y se expuso `POST /api/content/:contentId/listo`.

BUG-07: Subidas Multipart Huérfanas en Garage/S3 ante Fallos en BD

MEDIA

Problema: Si la inserción en base de datos fallaba después de invocar `createMultipartUpload` en S3/Garage, la sesión quedaba abierta indefinidamente consumiendo recursos en el almacenamiento.

Solución: Se encapsuló la persistencia en un bloque `try/catch` con llamada de rollback a `minioService.abortarMultipartUpload` ante cualquier excepción relacional.

BUG-08: Excepción no Controlada (HTTP 500) y Rango de Partes S3

MEDIA

Problema: En `/complete`, un body sin `parts` provocaba un `TypeError` no capturado (HTTP 500). Asimismo, `partNumber` no verificaba el rango permitido por S3 (1 a 10.000).

Solución: Validación de rango `1 <= partNumber <= 10000` y validación de existencia y no vacuidad de `parts` respondiendo con `400 Bad Request` controlado.

BUG-09: Aceptación de Fechas Inválidas (`Invalid Date`) en Programación

MEDIA

Problema: `marcarComoProgramado` aceptaba cadenas malformadas que resultaban en `NaN` en tiempo de ejecución.

Solución: Validación estricta con `isNaN(scheduledAt.getTime())` arrojando `BadRequestException` preventivo.

3. Implementación de la Historia de Usuario US-A6: Consulta de Catálogo

Historia de Usuario US-A6 (Sprint 3)

NUEVA FUNCIONALIDAD

Objetivo: Proveer un endpoint de consulta eficiente para catálogo de videos, con soporte para paginación (`page`, `size`) y filtrado por estado (`borrador`, `listo`, `programado`, `publicado`).

Optimización en BD: Se creó el índice `videos_status_idx` sobre la columna `status` de la tabla `videos` en PostgreSQL para acelerar búsquedas concurrentes.

Lógica de Negocio: Implementación de `ContentQueryService` con ejecución en paralelo (`Promise.all`) de la búsqueda y el conteo (`count()`), ordenamiento cronológico inverso (`created_at DESC`) y cálculo dinámico de páginas totales.

API Pública: Expuesto en `GET /api/content` y documentado en Swagger UI.

4. Tabla Comparativa: Estado Antes vs. Estado Actual

Área	Antes de la Corrección	Estado Actual
Seguridad	Riesgo de Path Traversal en nombres de archivo	Saneamiento con `basename` y bloqueo estricto (400)
Integridad de DTOs	Descarte silencioso de datos por `whitelist: true`	Validación declarativa tipada completa con `class-validator`
Almacenamiento S3	Multipart huérfano en Garage si BD fallaba; error 500 en partes	Abort automático garantizado; validación de rango (1..10000)
Consistencia de Estado	Estado `listo` solo en evento en memoria, no en PostgreSQL	Transacción ACID completa: `videos.status` y `metadata_versions`
Catálogo (US-A6)	Sin endpoint ni lógica de consulta implementada	`GET /api/content` paginado, filtrado y con índice B-Tree
Cobertura y Calidad	7 tests `it.todo` pendientes; advertencias de linter	**218 tests unitarios OK (0 pendientes)**, **60 E2E OK**, 0 lints

5. Decisiones Arquitectónicas (Planificación de Cierre de Sprint)

En acuerdo con la estrategia de equipo, los temas de arquitectura mayor señalados en la revisión técnica se programaron para el cierre del Sprint 3:

- **RabbitMQ y Patrón Outbox Transaccional:** Actualmente los eventos se despachan vía `EventEmitter2` en memoria. La transición hacia el broker RabbitMQ externo con Outbox Pattern se realizará al consolidar las historias de usuario de los módulos dependientes.
- **Desacoplamiento de Workers a Módulo 3:** El worker de publicación a YouTube permanece en modo stub para facilitar las pruebas del flujo de estados sin introducir acoplamientos prematuros.

6. Conclusión

El Módulo 1 ha sido completamente estabilizado y blindado contra fallos de datos y seguridad. Todos los endpoints se encuentran verificados en vivo mediante Docker y Swagger UI, listos para su integración en la rama `develop`.